



Software Engineering Fundamentals

Prepared by Mohammed Rafi

Computer Science Department
Northern Border University

Course Catalog

Programming Fundamentals

- Introduction to Python
- Data Structures & Algorithms
- Object-Oriented Programming

Web Development

- HTML, CSS & JavaScript
- React & Modern Frameworks
- Backend Development

Data Science

- Machine Learning Basics
- Data Analysis with Pandas
- Deep Learning Fundamentals

Cloud & DevOps

- AWS Essentials
- Docker & Kubernetes
- CI/CD Pipelines

The Nature of Software

- **Definition**

Software is a collection of **computer programs**, **procedures**, and **documentation** that perform specific tasks on a computer system.

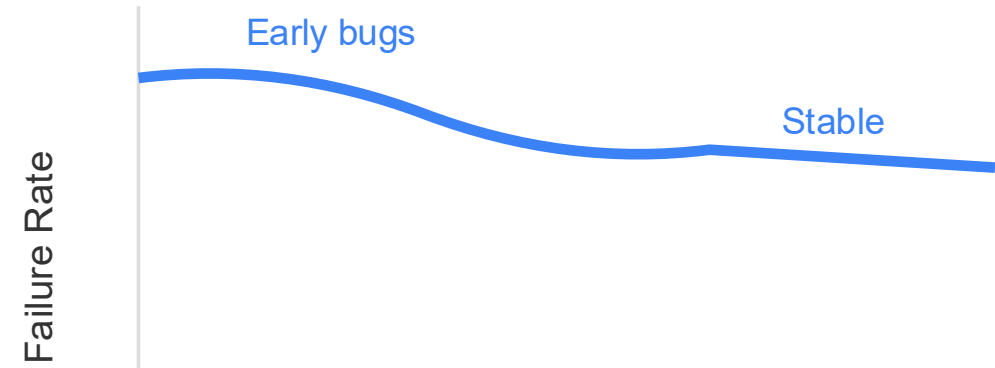
- **Key Characteristics**

- Intangible and logical in nature
- Does not wear out but can become obsolete
- Developed or engineered, not manufactured

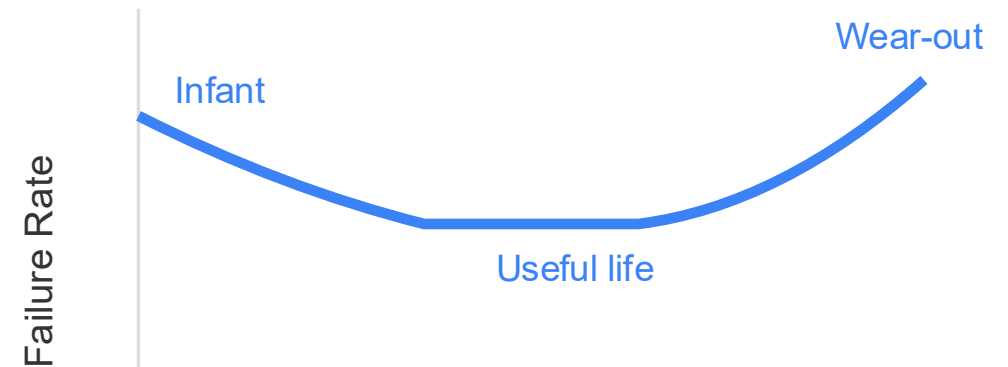
Key Differences

- ▶ **Software:** Logical components, programs, and data that run on hardware
- ▶ **Hardware:** Physical components that make up a computer system
- ▶ **Failure Pattern:** Software shows early failures (bugs), hardware shows wear-out failures
- ▶ **Maintenance:** Software can be updated remotely, hardware requires physical replacement
- ▶ **Degradation:** Software doesn't wear out but may become obsolete; hardware degrades over time
- ▶ **Replication:** Software can be copied infinitely; hardware must be manufactured

Software Failure Curve



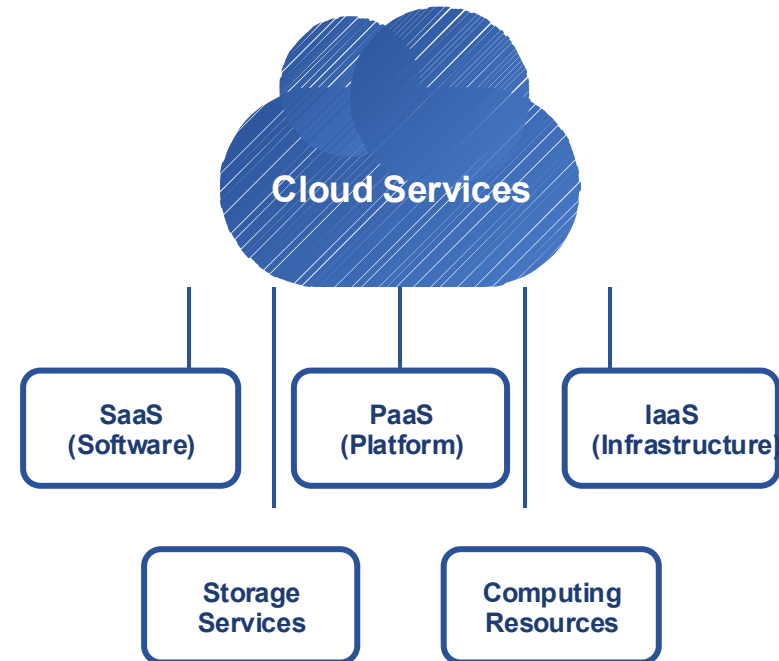
Hardware Failure Curve



- ▶ **System Software**
Operating systems, device drivers, and utilities
- ▶ **Application Software**
End-user programs for specific tasks
- ▶ **Engineering/Scientific Software**
CAD, simulation, and analysis tools
- ▶ **Embedded Software**
Software within hardware devices
- ▶ **Web Applications**
Browser-based software and services
- ▶ **Mobile Applications**
Software for smartphones and tablets

A t i f i l l t l i

Cloud Computing Architecture



Software Myths

Myths

- Adding more people speeds up delayed projects
- Software requirements can be fully specified upfront
- Once software is delivered, the job is done
- More documentation equals better quality

Reality

- Brooks's Law: Adding people increases complexity
- Requirements evolve throughout development
- Maintenance is 60-80% of software lifecycle
- Quality comes from good practices, not volume

Software Process Models

Waterfall

Sequential phases: requirements, design, implementation, testing, maintenance

Incremental

Develop and deliver system in increments, each adding functionality

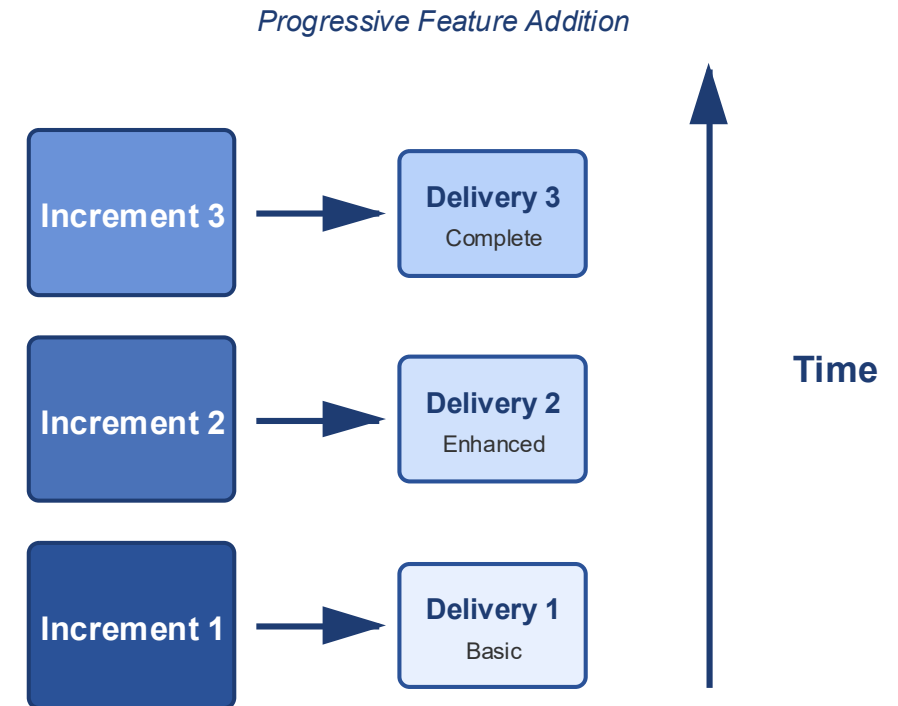
Formal Methods

Mathematical specification and verification of software systems

Agile

Iterative development with frequent customer feedback and adaptation

Incremental Development



Agile Development

Agile Principles

- Customer satisfaction through early and continuous delivery
- Welcome changing requirements, even late in development
- Deliver working software frequently
- Business and developers work together daily
- Build projects around motivated individuals
- Face-to-face conversation is the best communication

Agile Manifesto

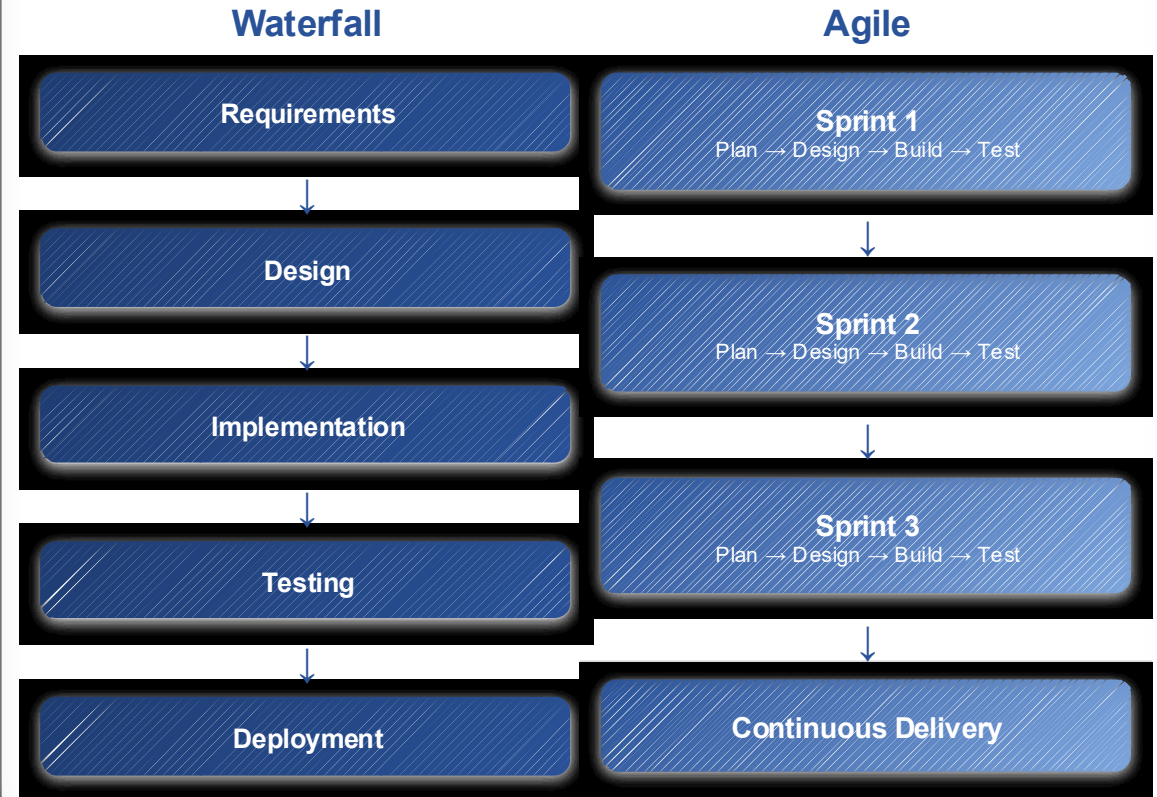
Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

Agile vs Waterfall



Requirements Engineering

User vs System Requirements

User Requirements

- Written for customers
- Natural language descriptions
- High-level statements
- Focus on what system should do

User Perspective

Business Goals

System Requirements

- Written for developers
- Detailed specifications
- Technical descriptions
- Focus on how system works

System Perspective

Technical Details

Functional vs Non-Functional Requirements

Functional Requirements

- Define what the system should do
- Describe specific behaviors or functions
- Examples: user login, data processing, report generation

Non-Functional Requirements

- Define how the system should perform
- Describe system qualities and constraints
- Examples: performance, security, scalability, usability

Functional

What the system does



Non-Functional

How well the system performs

UML Diagrams Overview

Structural Diagrams

Class Diagram

Shows classes and relationships

Object Diagram

Instance of class diagram

Component Diagram

System components

Deployment Diagram

Hardware topology

Package Diagram

Package dependencies

Composite Structure

Internal structure

Profile Diagram

UML extensions

Behavioral Diagrams

Use Case Diagram

System functionality

Activity Diagram

Workflow processes

State Machine

Object states

Sequence Diagram

Message interactions

Communication

Object interactions

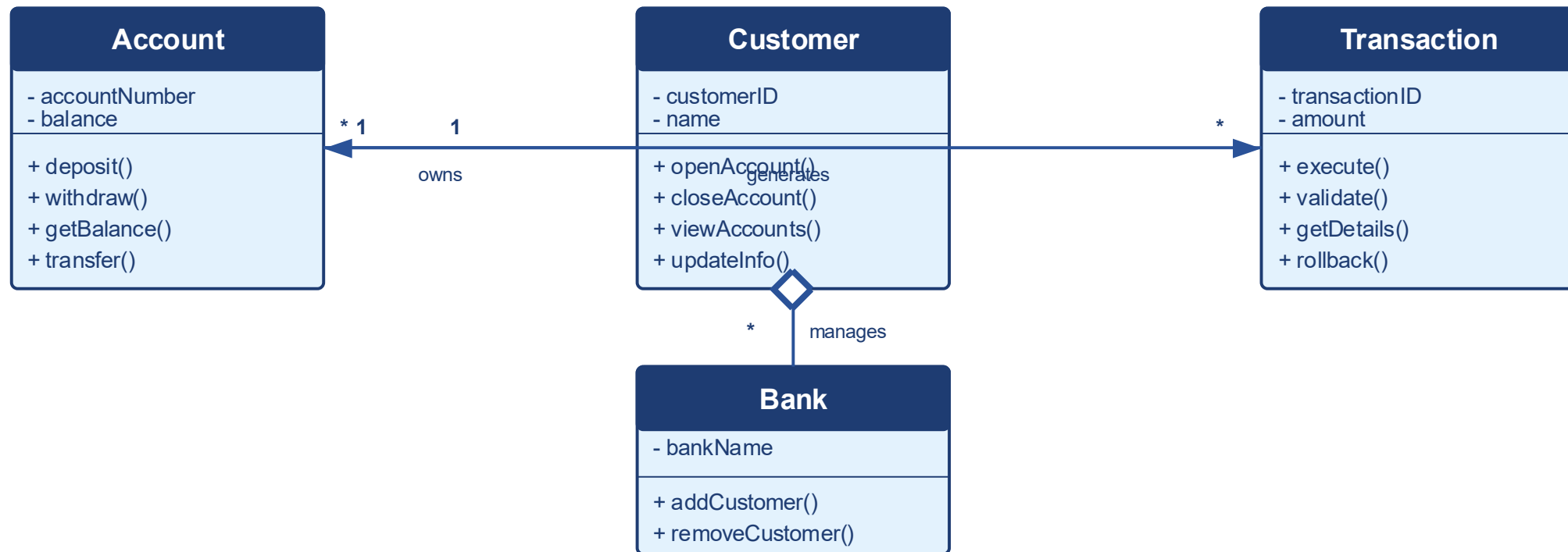
Timing Diagram

Time constraints

Interaction Overview

Interaction flow

Class Diagrams



Use Case Diagrams

Key Components

Actors

External entities that interact with the system (users, other systems)

Use Cases

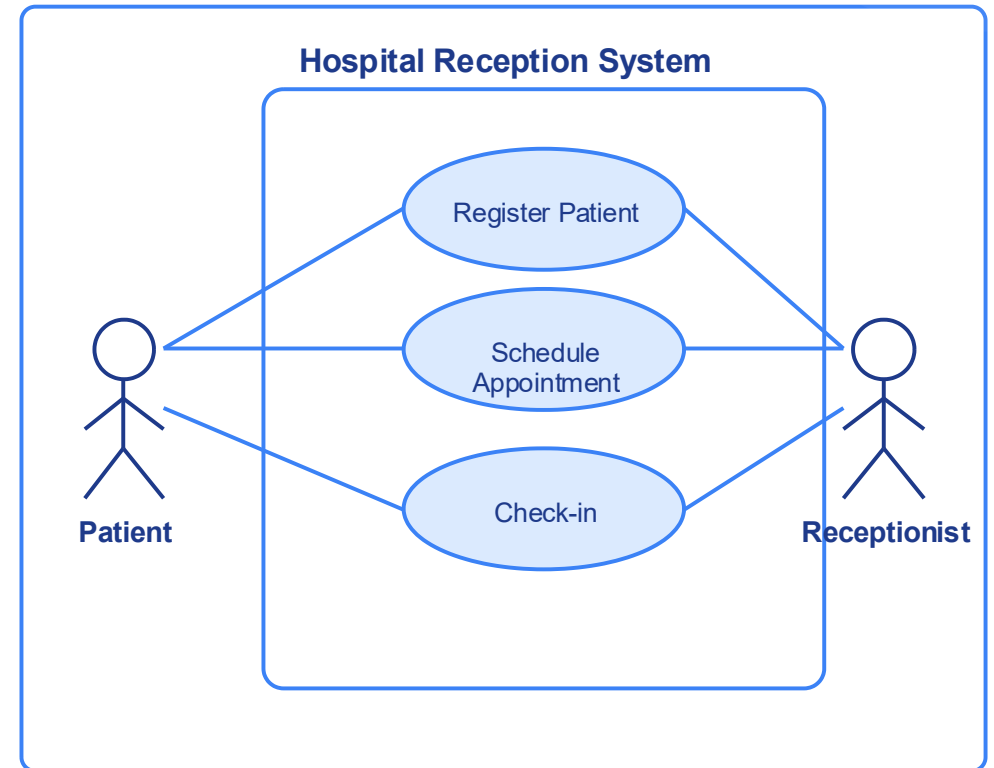
Specific functionalities or services the system provides

Relationships

Connections between actors and use cases showing interactions

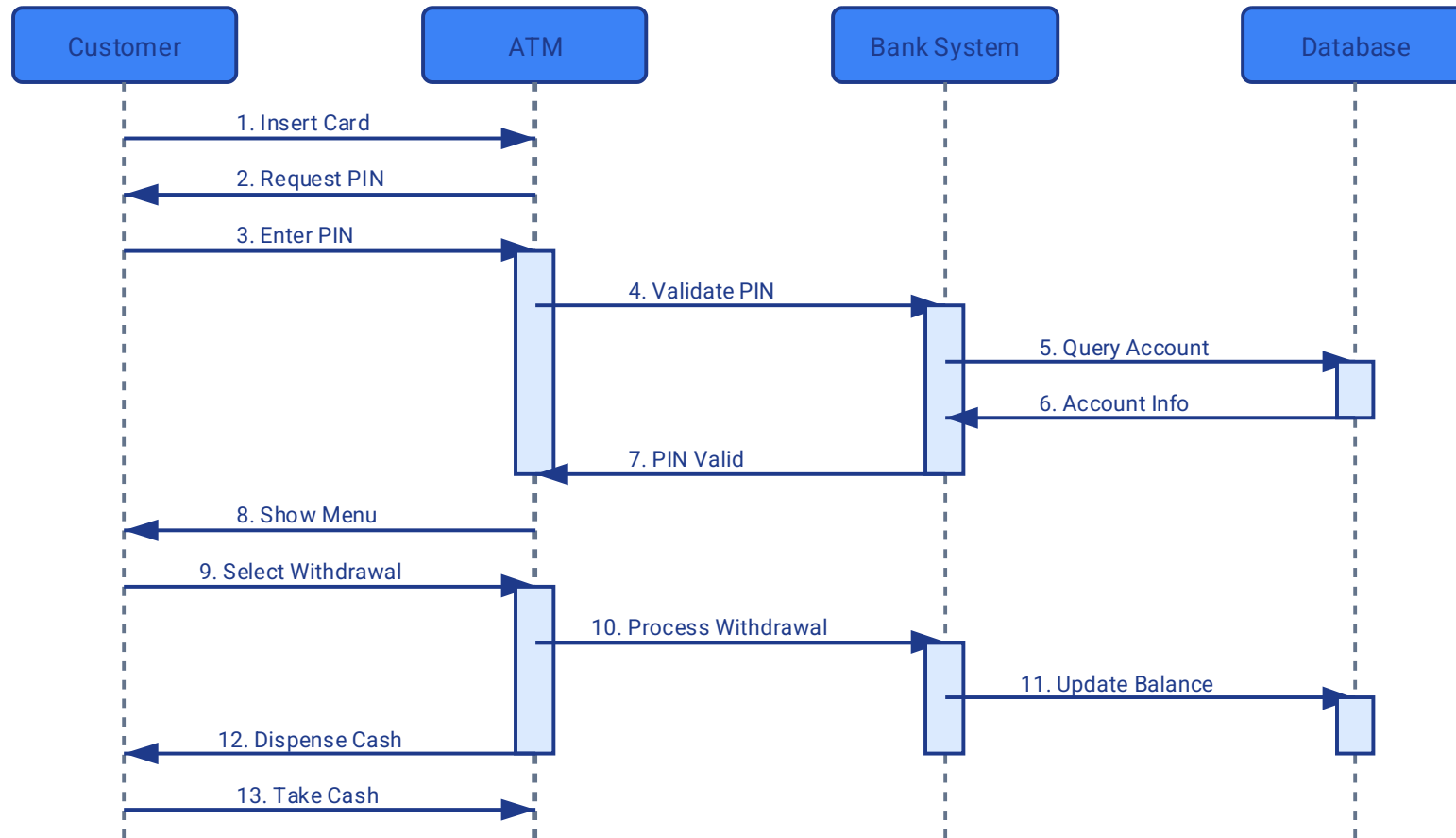
Use case diagrams visualize system functionality from the user's perspective

Hospital Reception Example



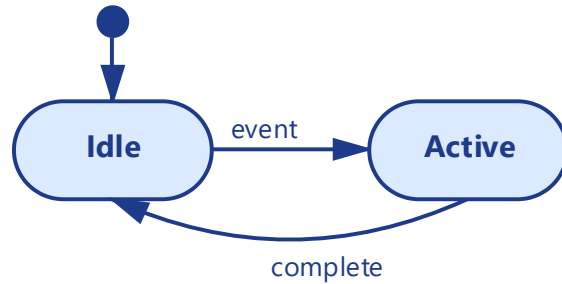
Sequence Diagrams

Banking Transaction Flow

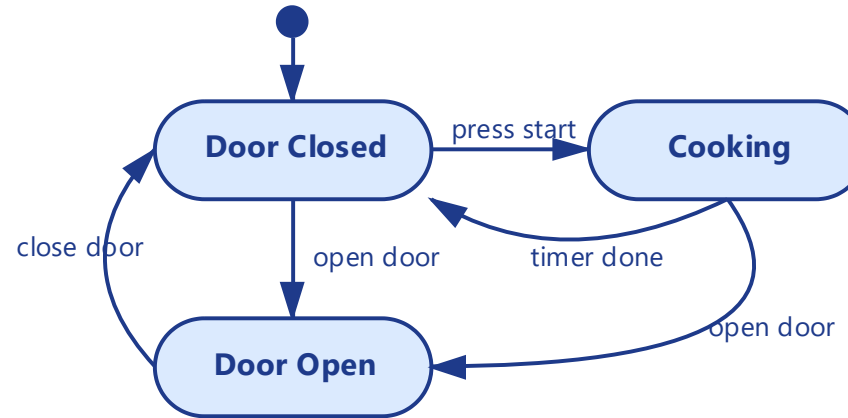


State Machine Diagrams

Basic State Machine



Microwave State Machine Example



Key Components

- Initial State: Starting point (filled circle)

State States: Conditions or situations (rounded rectangles)

→ Transitions: Events that trigger state changes (arrows)

Events: Triggers labeled on transitions

Guards: Conditions that must be true for transition [condition]

Usage Notes

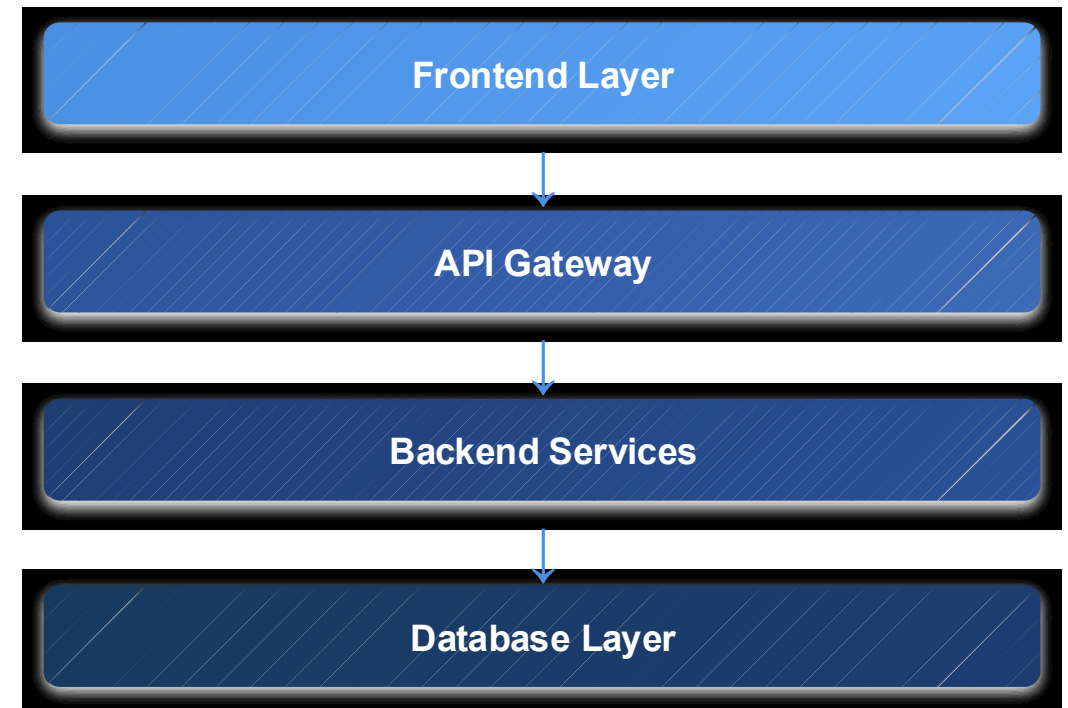
- Model object behavior and lifecycle
- Show how events affect state changes
- Useful for UI flows and protocols
- Can include nested states
- Support concurrent states
- Help identify edge cases

Architectural Design Overview

Key Components

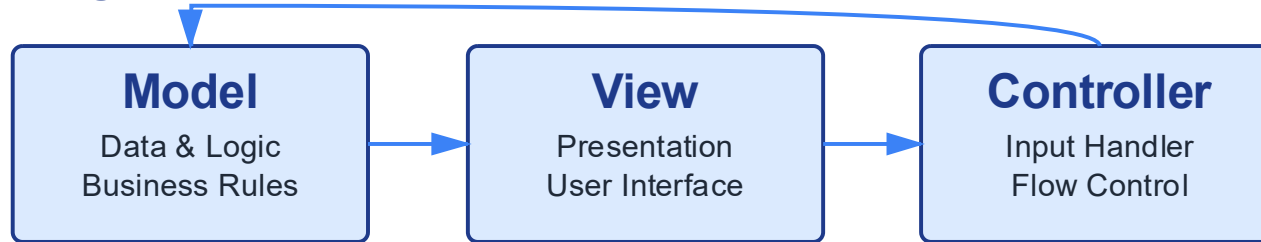
- **Frontend Layer:** User interface and interaction
- **Backend Services:** Business logic and data processing
- **Database Layer:** Data storage and management
- **API Gateway:** Request routing and authentication
- **Microservices:** Modular service architecture

System Architecture

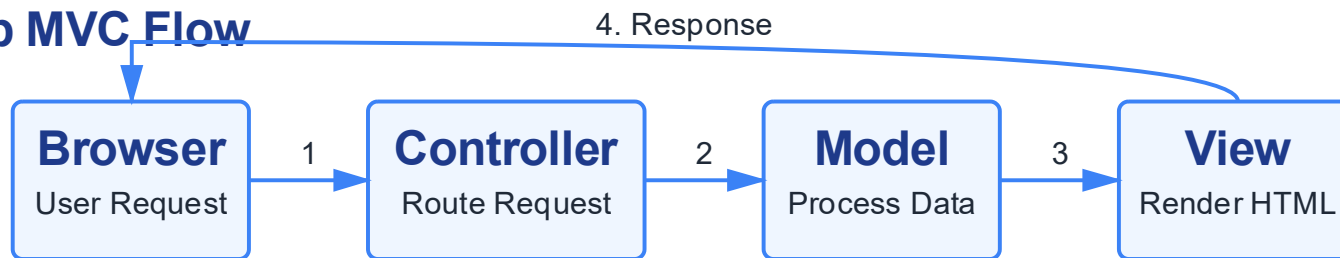


MVC Pattern

MVC Organization



Web MVC Flow



Key Benefits

- Separation of Concerns: Clear division between data, presentation, and logic
- Maintainability: Easier to update and modify individual components
- Testability: Components can be tested independently
- Reusability: Models and controllers can be reused across different views

Layered Architecture

Presentation Layer

User interface and interaction components

Application Logic Layer

Business rules and application workflows

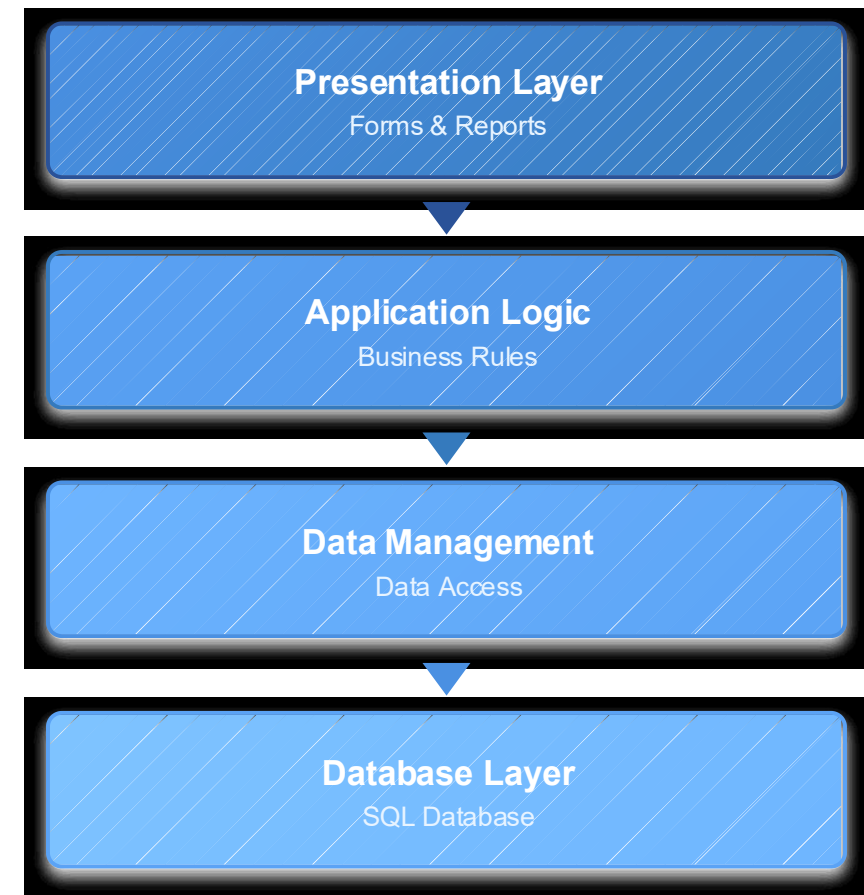
Data Management Layer

Data access and persistence operations

Database Layer

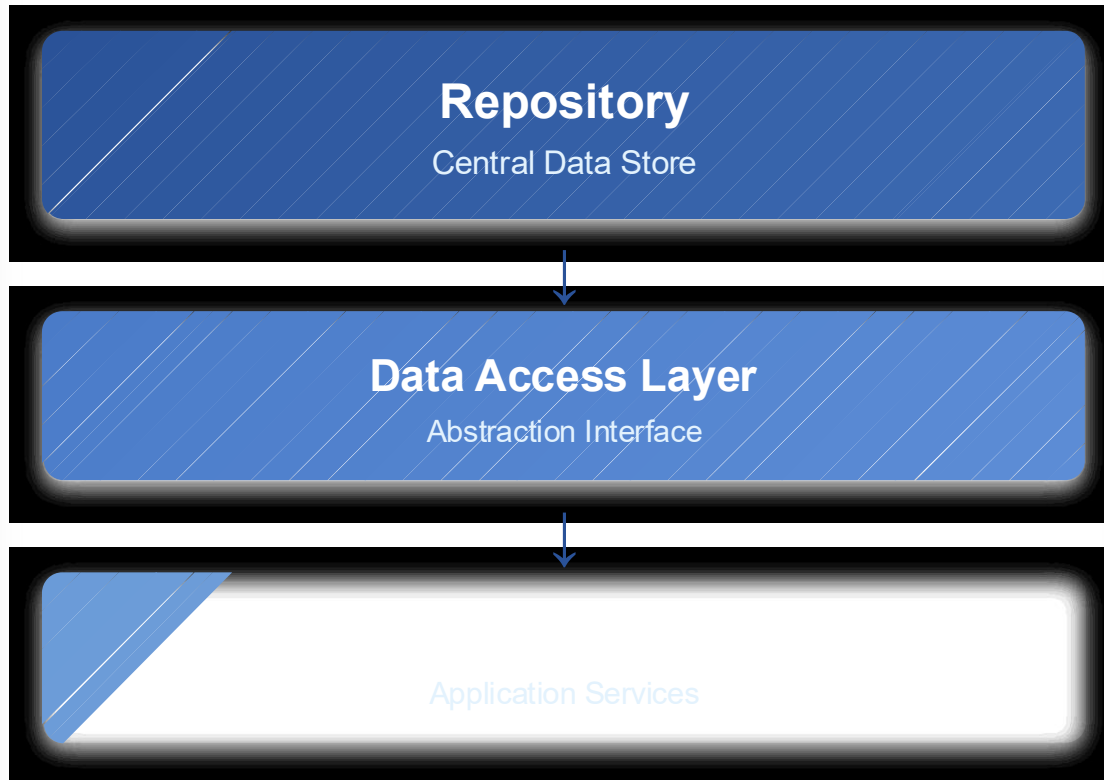
Physical data storage and retrieval

LIBSYS Architecture



Repository Architecture

Repository Model



IDE Integration

```
class RepositoryManager {
    public getData() {
        return this.repository;
    }
}
```

// Repository Pattern Benefits:

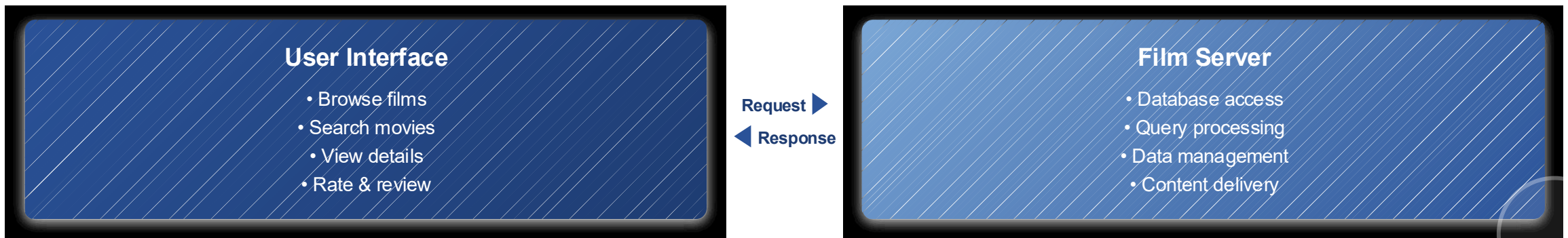
- Decouples business logic
- Centralizes data access
- Improves testability

Client-Server Architecture

Client-Server Model



Film Library Example

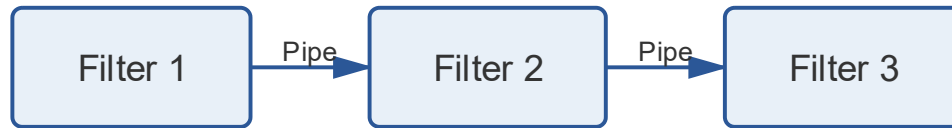


Pipe and Filter Architecture

Definition

- Data flows through a series of processing components (filters)
- Connected by pipes that transfer data between filters

Architecture Pattern



Example: Invoice Processing System



Key Characteristics

- Independent filters
- Reusable components
- Sequential processing
- Data transformation
- Easy to maintain

Benefits

- Modularity
- Flexibility
- Parallel processing
- Easy testing

Common Use Cases

- Data processing
- Compilers
- Image processing
- ETL systems

Transaction Processing Systems

Transaction Structure

1. Input

Data entry and validation

2. Processing

Execute business logic

3. Storage

Update database records

4. Output

Generate confirmation

ATM Example

Card & PIN Entry

Authentication process

Verify Balance

Check sufficient funds

Deduct Amount

Update account balance

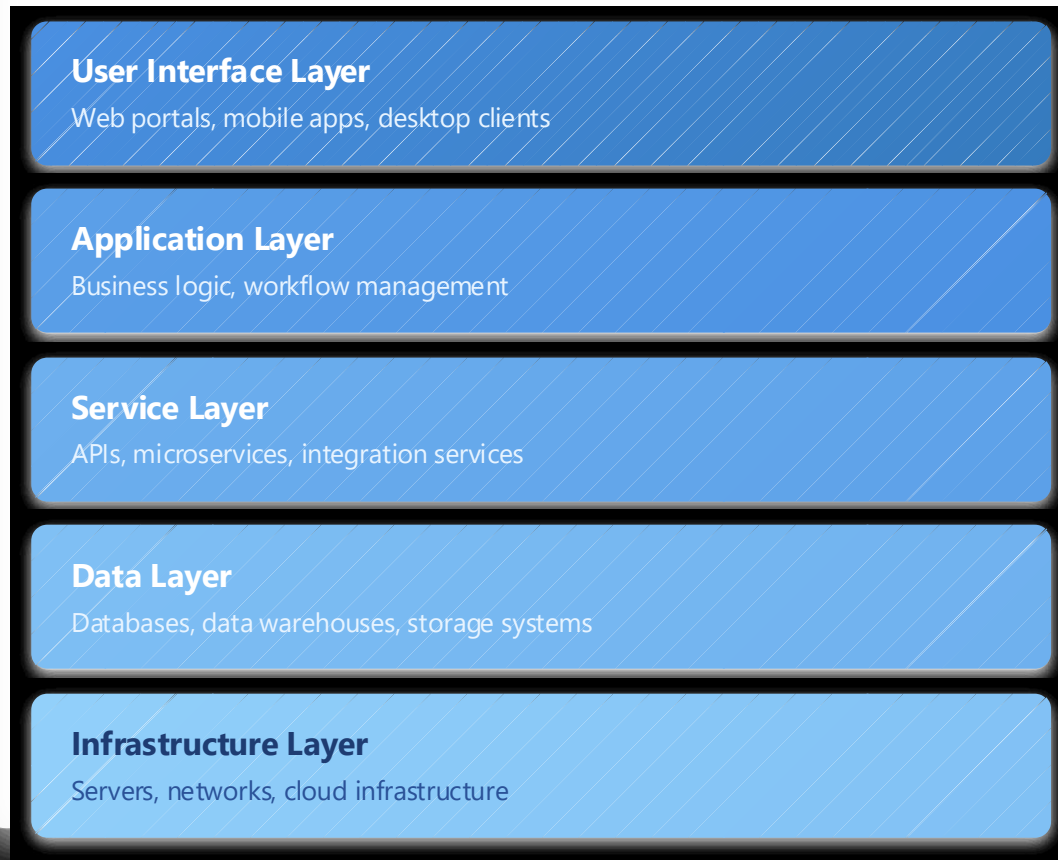
Dispense Cash

Print receipt

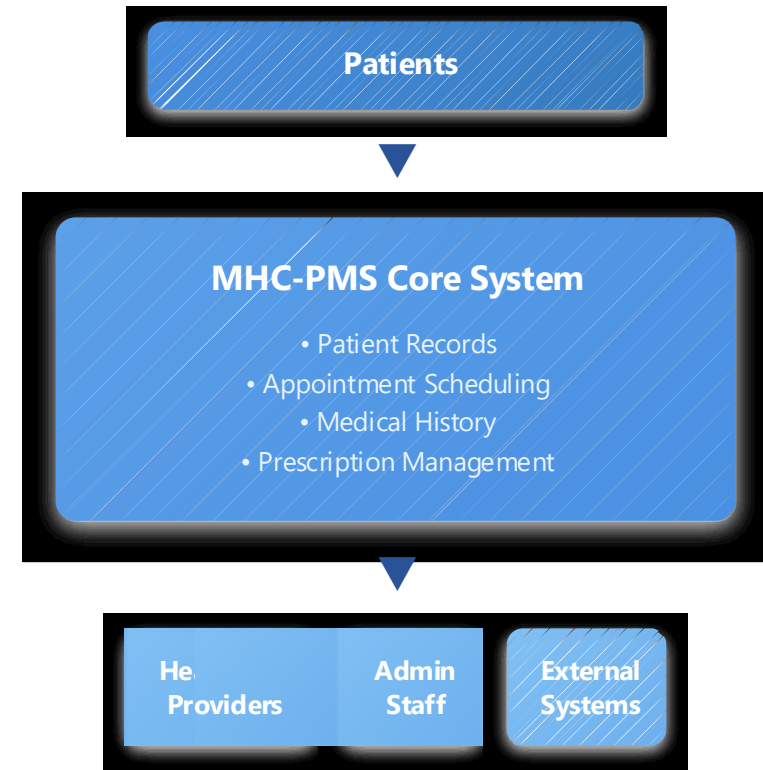
Real-time processing ensures ACID properties: Atomicity, Consistency, Isolation, Durability

Information Systems Architecture

Layered Information System Architecture



MHC-PMS Architecture



Key Features:

Centralized data management, role-based access control, real-time updates, and secure integration with external healthcare systems.

Language Processing Systems

Compiler Architectures

Traditional Compiler

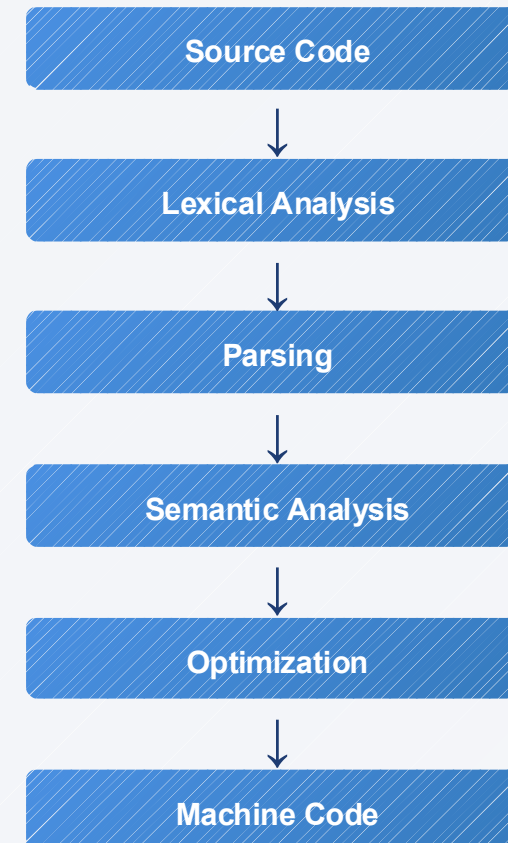
- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Intermediate Code Generation
- Code Optimization
- Code Generation

Modern Compiler

- Front-end Processing
- Middle-end Optimization
- Back-end Code Generation
- JIT Compilation

Processing Flow

Compilation Pipeline



Software Quality & V&V

Verification & Validation

Verification

"Are we building the product right?"

- Ensures software correctly implements specific functions
- Reviews, inspections, testing against specifications
- Static analysis, code reviews, unit testing

Validation

"Are we building the right product?"

- Ensures software meets user needs and requirements
- User acceptance testing, beta testing
- Validates against real-world scenarios

Key Differences

Verification: Process-oriented, checks conformance to specifications

Validation: Product-oriented, checks if it meets user expectations

CMM Maturity Levels

Level 5: Optimizing

Continuous process improvement

Level 4: Managed

Process measured and controlled

Level 3: Defined

Process characterized and understood

Level 2: Repeatable

Basic project management established

Level 1: Initial

Ad hoc, chaotic processes

CMM (Capability Maturity Model) provides a framework for improving software development processes through five progressive maturity levels

Software Testing Techniques

Unit Testing

Testing individual components or modules of software in isolation to ensure each unit functions correctly.

Acceptance Testing

Final testing phase to determine if the system satisfies business requirements and is ready for delivery.

Integration Testing

Verifying that different modules or services work together as expected when combined.

Regression Testing

Re-running tests after changes to ensure existing functionality still works correctly.

System Testing

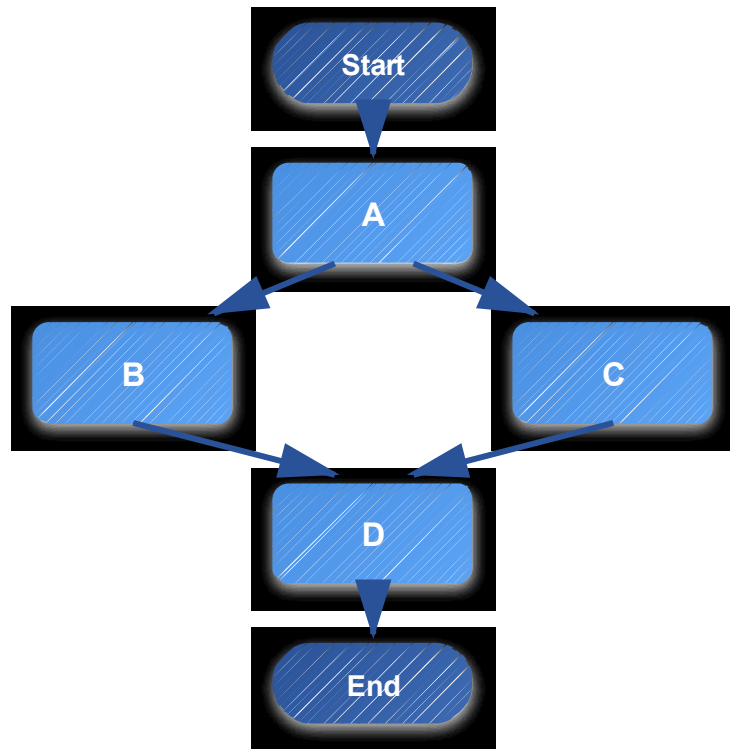
Complete testing of the entire system to validate it meets specified requirements.

Performance Testing

Evaluating system behavior under various load conditions to ensure speed, scalability, and stability.

Control Flow Testing

Control Flow Graph (CFG)



Path Selection Criteria

Statement Coverage

Execute every statement at least once

Branch Coverage

Execute every branch at least once

Path Coverage

Execute all possible paths through the code

Condition Coverage

Test all boolean sub-expressions

Test Planning & Estimation

Test Plan

- **Unit Testing:**
Developer-led component validation
- **Integration Testing:**
Module interaction verification
- **System Testing:**
End-to-end functionality checks
- **UAT:**
Client acceptance validation
- **Performance Testing:**
Load and stress analysis

LOC Estimation

- **Frontend:**
~15,000 lines (React components)
- **Backend:**
~20,000 lines (API services)
- **Database:**
~3,000 lines (SQL scripts)
- **Testing:**
~8,000 lines (test suites)

Total: ~46,000 LOC

Project Management - The 4 Ps

People

Team members, stakeholders, and resources involved in the project

Product

The deliverable or outcome that the project aims to create

Process

The methodology and workflows used to execute the project

Project

The overall initiative with defined goals, timeline, and scope

Thank You

Questions & Answers